

# 深度学习

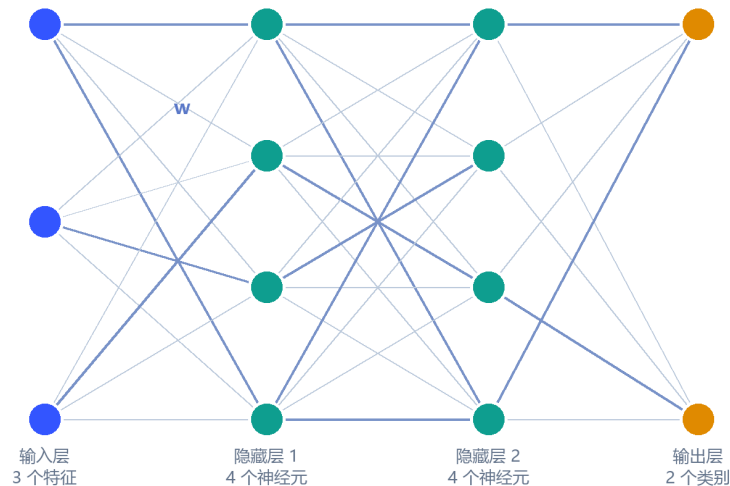
从「一个神经元」到「会自己找特征」

## 这次课要回答的三个问题

是什么 深度学习在学什么？—— 学「表示」，而不是学规则

怎么训 网络怎么从数据里把参数调出来？—— 前向、损失、更新。

为什么行 为什么深、为什么非线性、为什么 CNN 适合图像？



同一件事,矩阵写法  
 $h1 = \sigma(W1 \cdot x + b1)$

$W1 \ (4 \times 3) \cdot x \ (3 \times 1) + b$

把每一层看成一次「矩阵乘法 + 加偏置 + 非线性」，网络就成了可导的复合函数——这正是能用梯度下降训练它的原因。

预计授课时长 10 分钟 · 面向零基础初学者

# 深度学习为什么会突然“行”了？

## 2012 · AlexNet

ImageNet 图像分类  
错误率 26% → 15%

## 2017 · Transformer

注意力机制  
统一 NLP / 视觉 / 语音

## 2022 之后 · 大模型

预训练 + 微调  
一个范式覆盖几乎所有任务

**真正变了的不是算法有多复杂，而是我们不再手写规则，而是把「找规律」这件事交给数据和梯度。**

同样的神经网络结构，1950 年代就有雏形 —— 它等了三样东西才真正可用：

**海量数据 · GPU 算力 · 能被稳定训练的方法**

一句话总结：不是我们更会写规则了，而是我们不再写规则了。

# 深度学习 = 让网络自己学出「表示」

传统机器学习：人来设计特征 → 模型做分类

深度学习：网络自己从数据里学出「从低级到高级」的多层表示



「特征从哪来」被改写了：不再由人设计，而是由数据一层层学出来。

# 它擅长什么，又不擅长什么

## 擅长

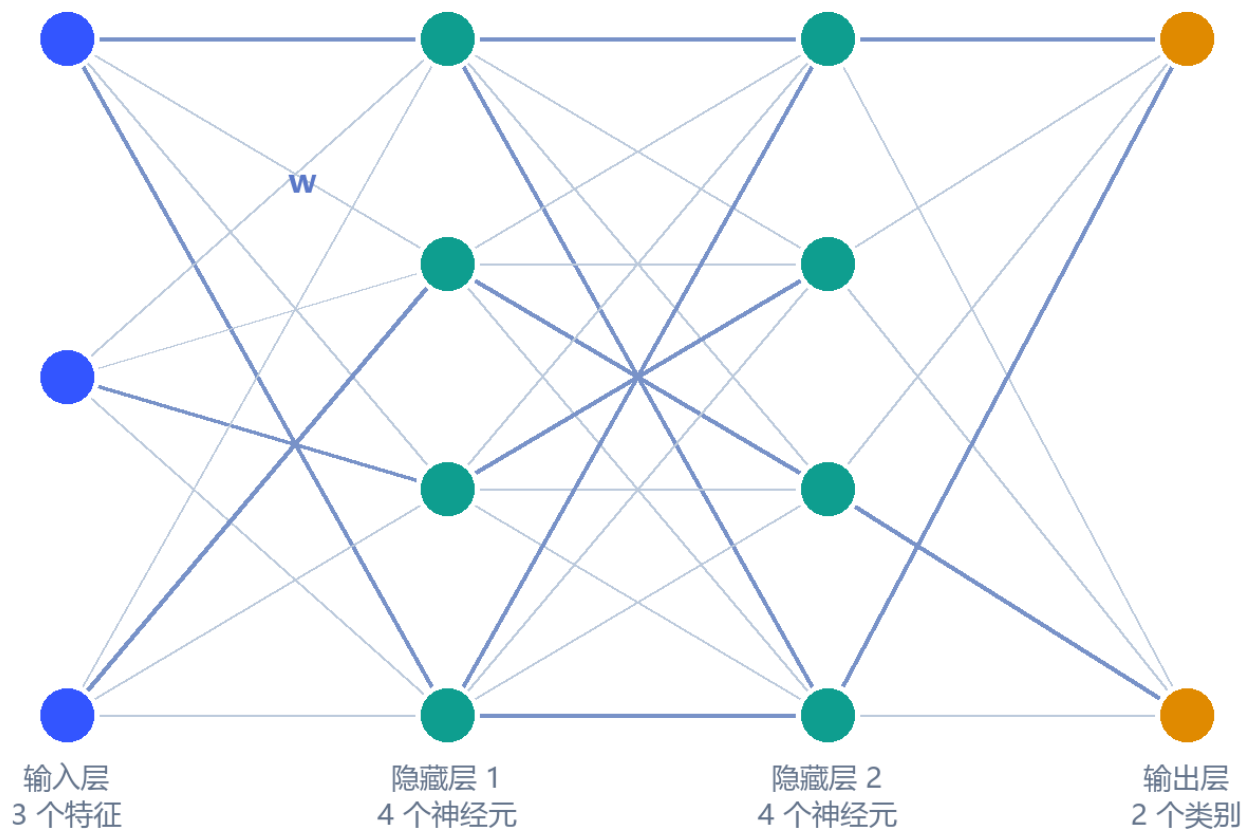
- 感知类任务：图像分类、目标检测、语音、翻译
- 有大量标注数据，且模式稳定、可重复出现
- 输入是高维、结构化很弱的信号（像素、波形、文本）

## 不擅长 / 要小心

- 需要严格可解释、可审计的决策（医疗、司法）
- 数据极少，或分布随时间明显漂移的场景
- 需要多步严密逻辑推理的任务，目前仍不稳定

**判断标准：如果「人看一眼就能给出标签」，而且这种样本能拿到成千上万条 —— 深度学习通常非常好用。反过来就要格外警惕。**

# 一个神经网络 = 层叠的加权求和 + 非线性



同一件事, 矩阵写法

$$h1 = \sigma(W1 \cdot x + b1)$$

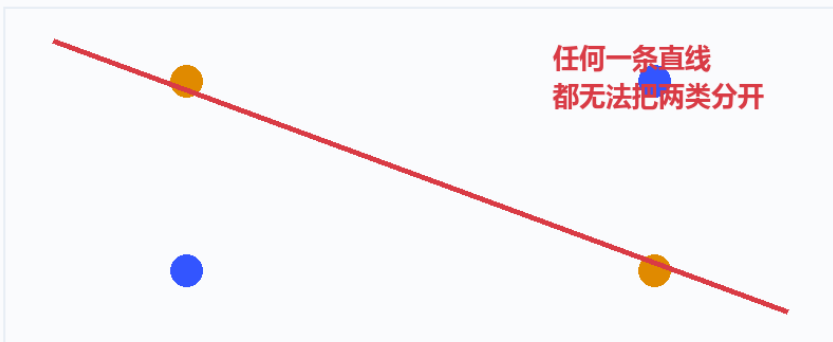
$W1 \ (4 \times 3) \cdot x \ (3 \times 1) + b$

把每一层看成一次「矩阵乘法 + 加偏置 + 非线性」，网络就成了可导的复合函数——这正是能用梯度下降训练它的原因。

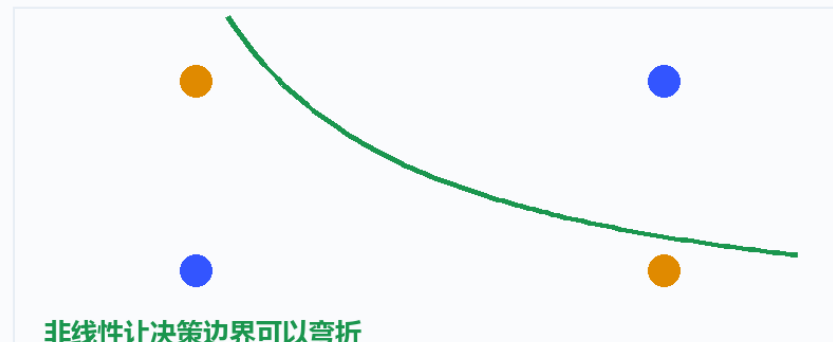
每一层都是一次「矩阵乘法 + 加偏置 + 非线性」——所以整个网络是可导的复合函数。

# 如果没有非线性，深层就白堆了

只用线性：分不开 XOR



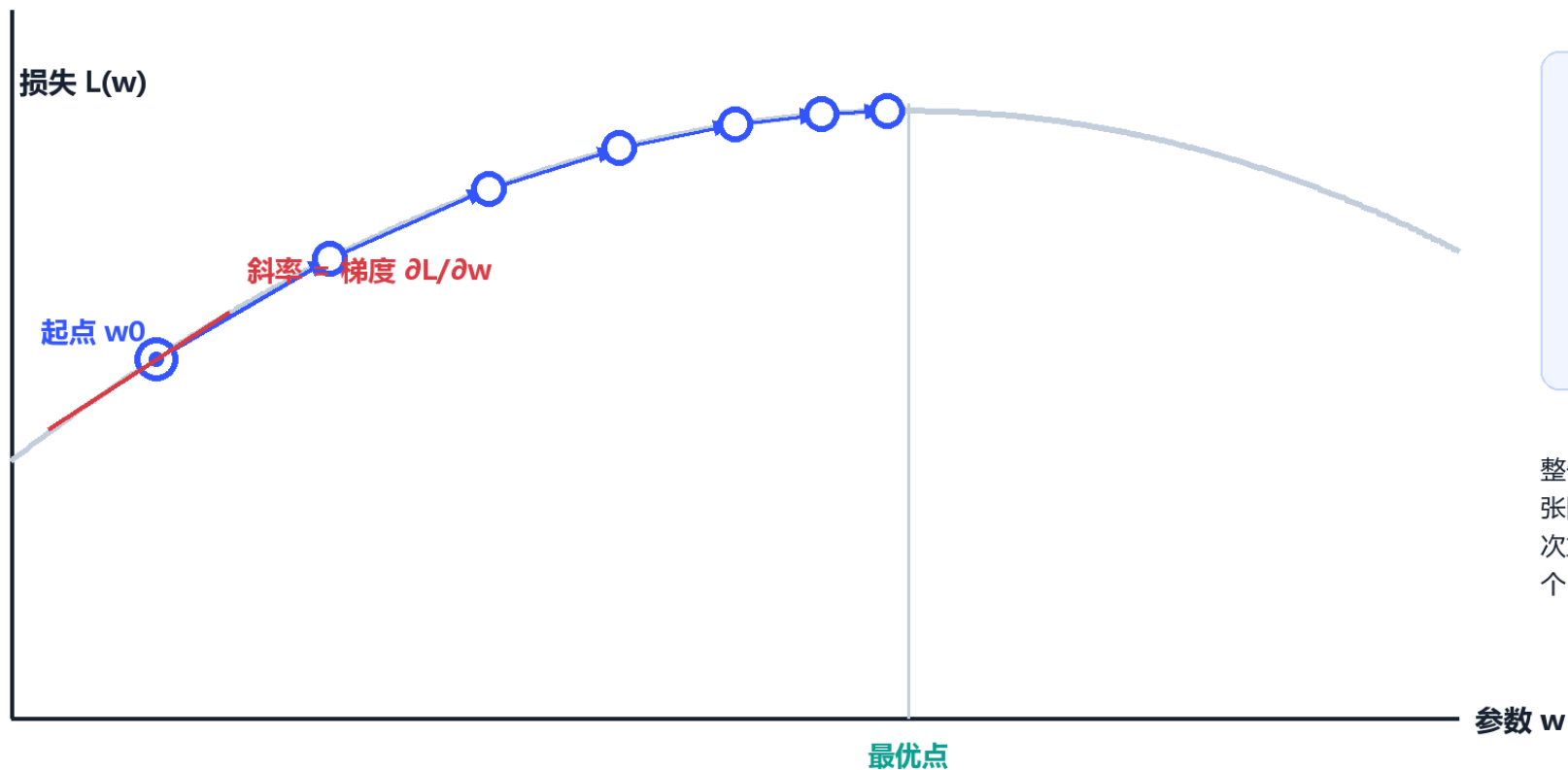
加上激活函数：可以弯曲空间



$\text{ReLU}(z) = \max(0, z)$  •  $\text{sigmoid}(z) = 1/(1 + e^{(-z)})$  —— 「矩阵再怎么乘还是线性，只有激活函数才能表达复杂函数」

两层线性叠起来还是线性 —— 堆再深也等价于一层。这就是 1969 年 XOR 让研究停滞的原因。

# 训练 = 沿梯度方向一步步下坡



## 参数更新规则

$$w \leftarrow w - \eta \cdot$$

$$\partial L / \partial w$$

$\eta$ : 学习率

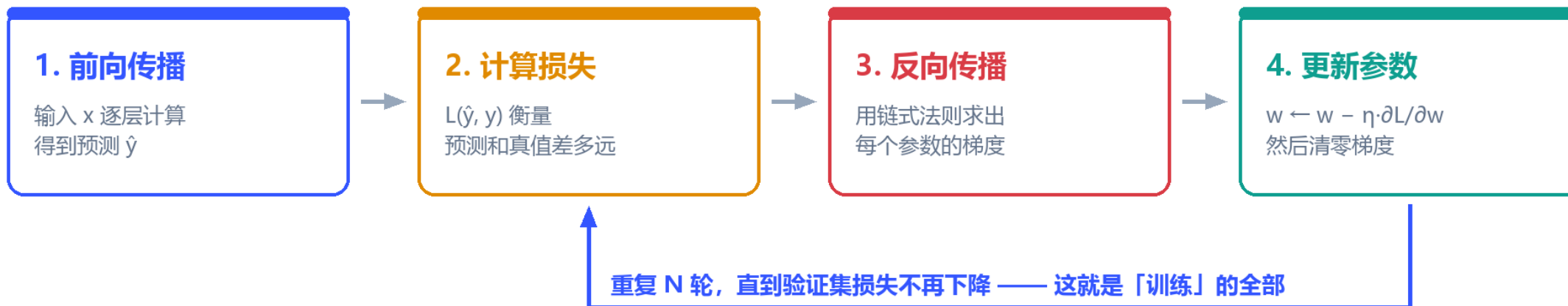
步子太小  $\rightarrow$  收敛慢

步子太大  $\rightarrow$  震荡/发散

整个深度学习训练，就是这张图在多维空间里的上百万次重复——参数不是百万个，而是上亿个。

更新公式:  $w \leftarrow w - \eta \cdot \partial L / \partial w$  —— 整个深度学习训练，就是这张图上百万次重复。

## 训练循环：四个动作不断重复



推理时只剩第一步：一次前向 = 一次预测 —— 没有损失、没有反向、没有更新。所以「训练慢、推理快」。

训练慢、推理快：推理只需要第一拍「前向」，没有损失、没有反向、没有更新。

# 损失函数定方向，反向传播算梯度

损失函数是一个标量：它给「这一次预测有多差」打分。

## 为什么用梯度下降，而不是直接解？

参数动辄上亿个，解析解根本写不出来；梯度只要「局部信息」，一次反向传播就能拿到全部参数的梯度。

### 交叉熵 Cross-Entropy

分类任务的标准选择

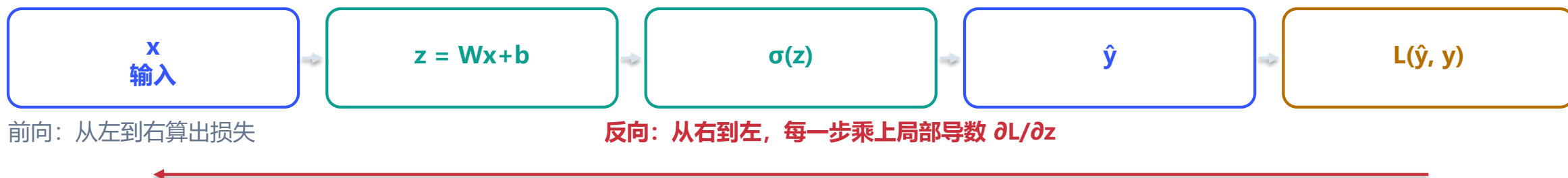
### 均方误差 MSE

回归任务：预测连续值

### 对比 / 三元组损失

让相似的样本在表示空间里靠近

反向传播 = 链式法则的高效实现：从输出端把误差一层层往回“摊”给每个参数。



**每一步的梯度 = 上游梯度 × 本步局部导数 —— 这就是所有框架「自动求导」的原理。**

# 决定「每一步怎么走」的东西

梯度只给出方向，具体怎么迈这一步，由优化器决定。

## SGD

最朴素的梯度下降：每步只看一个 mini-batch

## 动量 Momentum

累积历史方向，减少震荡、加速穿过平坦区

## Adam / AdamW

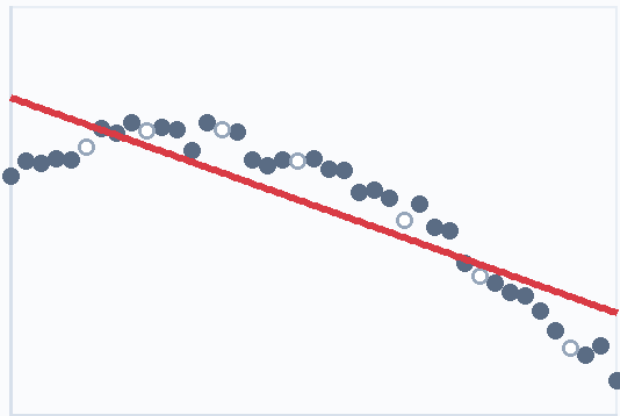
自适应地为每个参数调步长 —— 目前最常用的默认选择

**学习率是最重要的超参数：常用做法是先用较大学习率快速下降，再逐步衰减（cosine / step 调度），后期精细收敛。**

# 训练得好 $\neq$ 学得好：欠拟合 / 过拟合

## 欠拟合 Underfitting

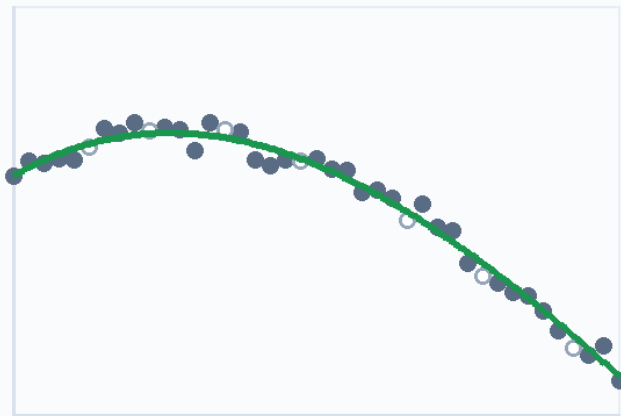
模型太简单，训练集都学不好



训练误差 高 测试误差 高

## 刚好 Just right

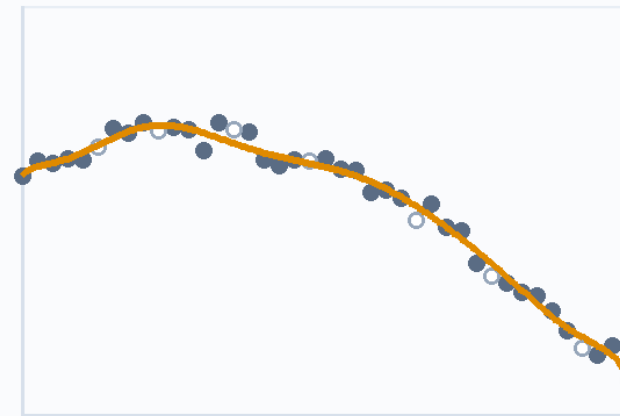
复杂度与数据量匹配



训练误差 低 测试误差 低

## 过拟合 Overfitting

死记训练集，没学到规律



训练误差 极低 测试误差 高

# 对付过拟合：一套组合拳

## 更多数据 / 数据增强

最有效，也最容易被忽略：旋转、裁剪、加噪都是廉价的新样本

## 正则化

L2 权重衰减、Dropout：限制模型「记死答案」的能力

## 早停 Early Stopping

验证集损失不再下降就停下，别把训练集背下来

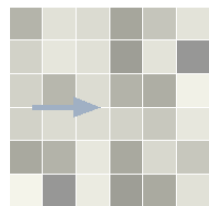
## 更简单的模型

参数少一点、层浅一点；容量要和数据量匹配

**训练集用来学，验证集用来选，测试集只用一次 —— 绝不能拿测试集调参。**

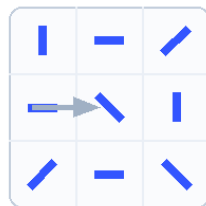
这是初学者最常犯、后果最严重的一个错误：测试集一旦被反复使用，它就不再是「没见过数据」。

# 卷积网络为什么适合图像：分层地“从像素到物体”



**原始像素**

一堆数字，没有人看得出是什么



**浅层：边缘 / 方向**

检测器只看局部小窗口



**中层：部件 / 纹理**

把边缘组合成有意义的形状



**深层：物体（猫）**

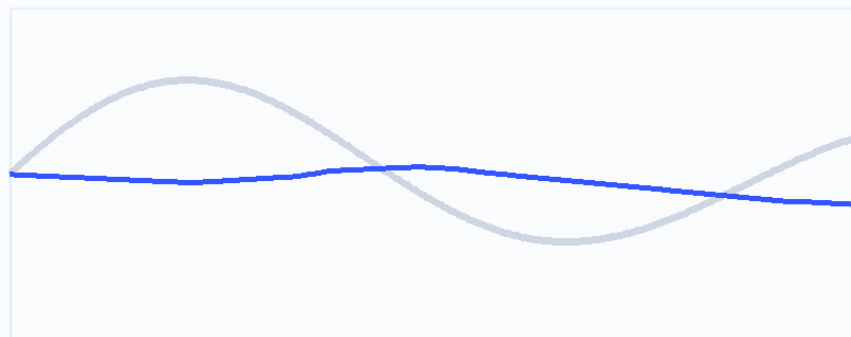
组合部件，得到类别判断

对照：传统方法要靠人设计 SIFT / HOG 等特征；深度学习把「特征工程」也交给了数据。

**三条设计：局部连接、权重共享、逐层堆叠 —— 参数量因此远小于全连接网络。**

## 深度为什么重要：同样的参数预算，深比宽更「会拐弯」

浅而宽 · 约 12 个 ReLU 单元



折点少 → 只能粗略跟随

深而窄 · 约 12 个 ReLU 单元



折点多 → 能拼出陡峭变化

— 真实目标函数      — 模型拟合结果 (ReLU 单元的线性组合)

同样的参数预算，深度换来的是「组合能力」：浅层直接拟合函数，深层逐级复用中间结果。

# 值得记住的四件事

## 数据

先看数据：类别是否均衡、有没有泄漏、标签是否干净。数据有问题，模型再好也没用。

## 基线

先跑一个最简单的基线（逻辑回归 / 小网络），有了对照才知道深度网络到底带来了多少提升。

## 监控

同时记录训练损失和验证损失、以及关键指标；只看 loss 下降会错过过拟合。

## 可控

固定随机种子、记录配置文件、别一次改太多变量 —— 否则无法归因。

**经验之谈：深度学习里 90% 的失败不是模型不够深，而是数据、标签或评估方式出了问题。**

# 20 行代码：从数据到训练循环

PyTorch

```
import torch
from torch import nn

# 1. 数据: 一个 mini-batch
x = torch.randn(64, 784)      # 64 张 28x28 的图
y = torch.randint(0, 10, (64,)) # 10 个类别的标签

# 2. 模型: 三层 MLP
model = nn.Sequential(
    nn.Linear(784, 256), nn.ReLU(),
    nn.Linear(256, 64),  nn.ReLU(),
    nn.Linear(64, 10),
)

# 3. 损失 + 优化器
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.AdamW(model.parameters(), lr=1e-3)

# 4. 训练循环: 正是前面那四拍
for step in range(1000):
    y_hat = model(x)          # 前向
    loss = loss_fn(y_hat, y)  # 算损失
    opt.zero_grad()          # 清空旧梯度
    loss.backward()           # 反向传播
    opt.step()                # 更新参数
```

实践

## 对着代码看四拍

### 前向传播

`y_hat = model(x)`

### 计算损失

`loss_fn(y_hat, y)`

### 反向传播

`loss.backward()`

### 更新参数

`opt.step()`

**注意 `opt.zero_grad()`：梯度默认累加，不清零会越来越大的。**

# 带走这四句话就够了

## 1. 深度学习学的是「表示」

不是人手写规则，而是让网络从数据里一层层学出特征。

## 2. 训练是四拍循环

前向 → 损失 → 反向 → 更新，反复迭代，直到验证集不再变好。

## 3. 非线性是深度的前提

没有激活函数，再深的网络也只是一次线性变换。

## 4. 关键在泛化

训练误差低没有意义，新数据上的表现才算数。

**如果只记一句：深度学习把「特征工程」这件苦活，交给了数据和梯度。**

下一步建议：动手跑一遍上面 20 行代码，然后把模型加宽、加深、去掉 ReLU，观察各自的变化。